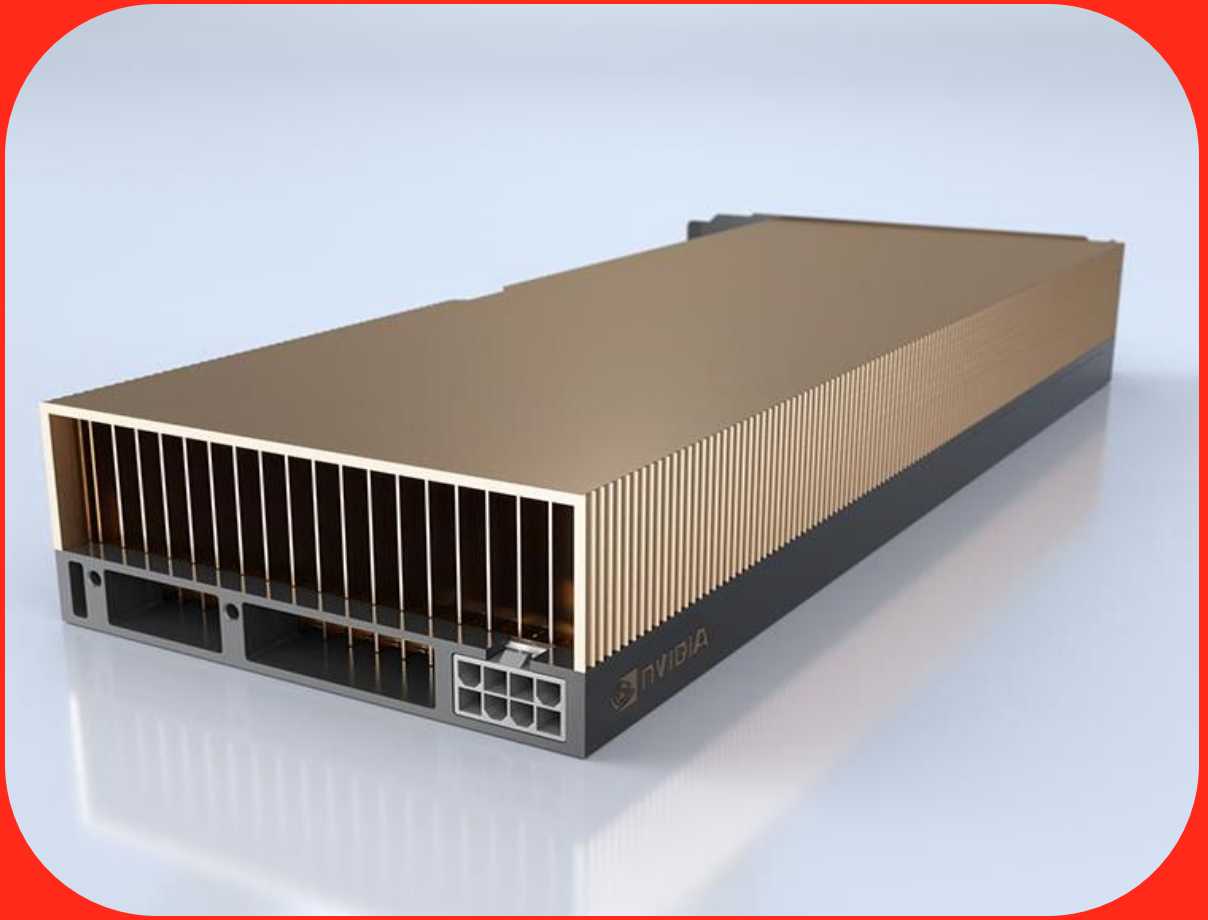
How to use an NVIDIA A40



UAB
Universitat Autònoma de Barcelona

Whats on the menu

- `Trainer` and its arguments
- Memory and LLM training
- Mixed precision
- `torch.compile`
- Gradient Checkpointing
- Dataloading
- Optimizer States
- FlashAttention
- LoRA
- QLoRa

Our practical case:

- Summarization task
- NVIDIA A40 x1
- **Qwen3 0.6 (Base)**
- Batch size = 2
- Seq len = 2048

GitHub:

- Code examples for everything!

The two ways

I will try to showcase code examples for both

"Torch native"

- Better to learn, more control
- Applicable to something that isnt LLMs

transformers.Trainer

- Already seen in class
- Easier to set up for simple tasks

TORCH VERSION: 2.10.0

transformers.Trainer

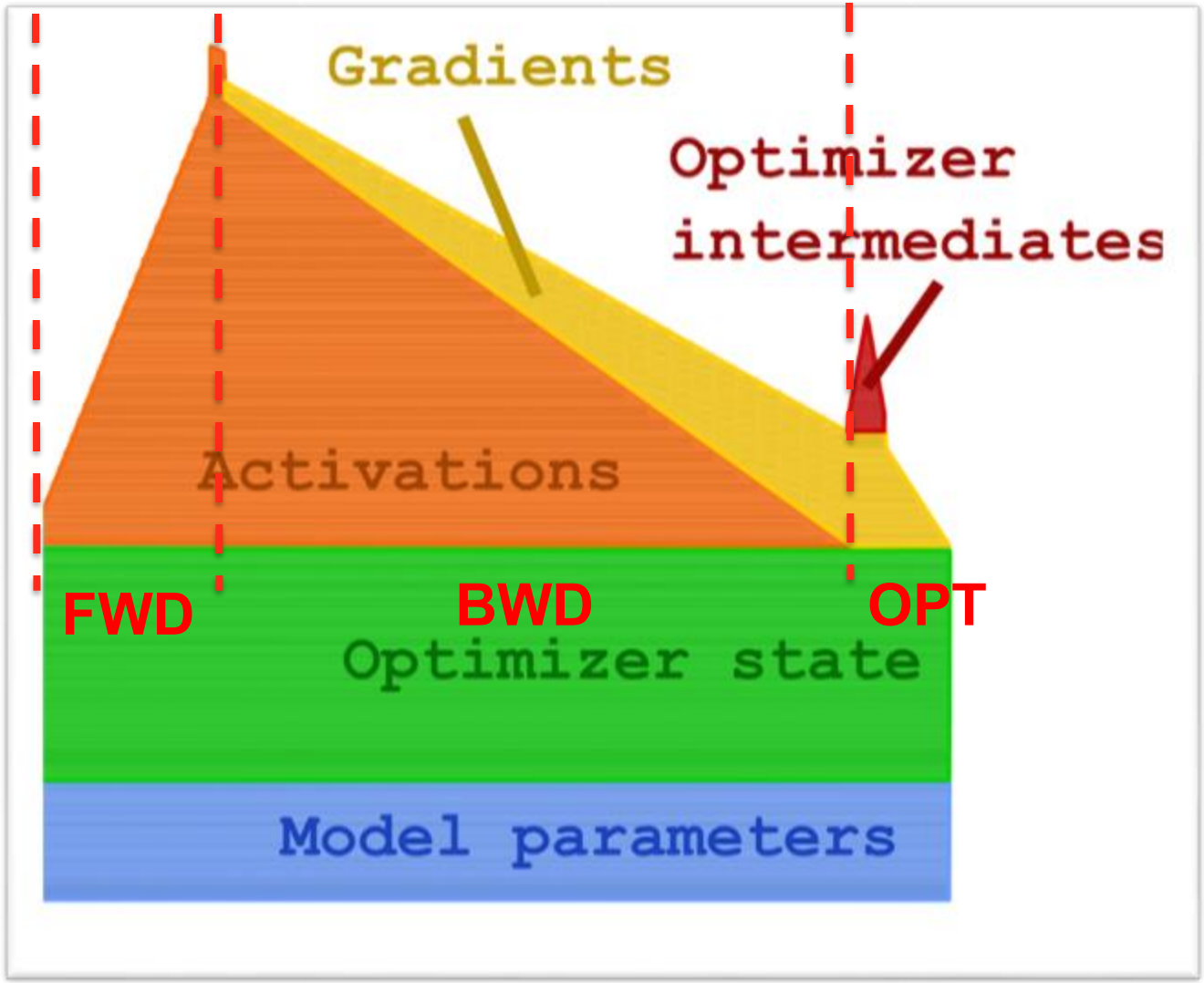
- You know it, you love it

Logging (that we care about):

- Tokens per second
- Samples per second
- Peak memory

```
args = TrainingArguments(
        output_dir="./checkpoints",
        # --- throughput ---
        per_device_train_batch_size=1,
        gradient_accumulation_steps=1,
        # --- precision ---
        bf16=True,
        tf32=True,
        # --- compilation ---
        torch_compile=True,
        # --- schedule ---
        max_steps=2000,
        warmup_steps=100,
        learning_rate=2e-5,
        lr_scheduler_type="cosine",
        weight_decay=0.01,
        # --- logging & saving ---
        logging_steps=50,
        eval_strategy="steps",
        eval_steps=500,
        save_steps=500,
        save_total_limit=2,
        # --- misc ---
        dataloader_num_workers=3,
        dataloader_pin_memory=True,
        report_to="none",
)

trainer = Trainer(model, args)
trainer.train()
```

Memory during training



See more here:
- pytorch blog
- Hf blog

Mixed Precision

- Using this simple trick reduce **your memory by HALF**!
- No difference in the loss

With native torch:
`model = model.to(torch.bfloat16)`

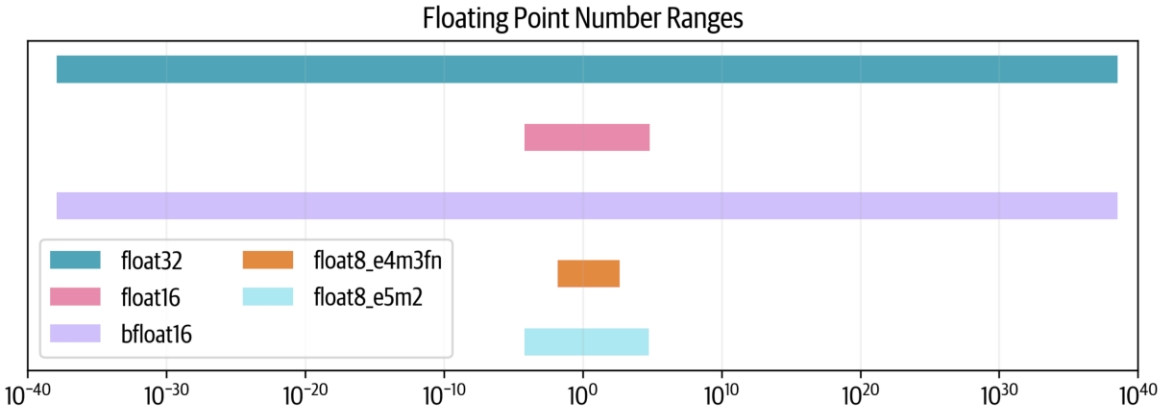
With `Trainer`:
`bf16=True` in the args

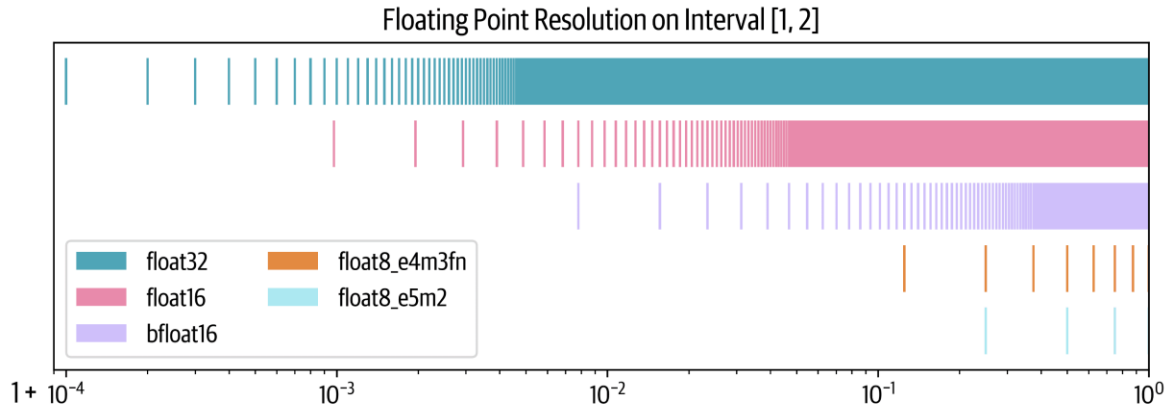
```
  Memory profile – single training step
=================================================================
 precision   μ   bef-fwd   aft-fwd    pk-fwd   aft-bwd    pk-bwd  (GB)
-----------------------------------------------------------------
FLOAT  32    1     6.678    23.938    26.257     8.899    28.575
-----------------------------------------------------------------
BFLOAT 16    1     3.347    14.247    17.725     4.458    18.884
=================================================================
  bef-fwd  weights + optimizer states
  aft-fwd  + activations cached for bwd
  pk-fwd   + intermediate tensors during fwd
  aft-bwd  + gradients, - activations
  pk-bwd   worst case during bwd pass
=================================================================
```

Why **B**FLOAT16?

- Same range as float32
- Thus, no re-scaling!

- But less resolution





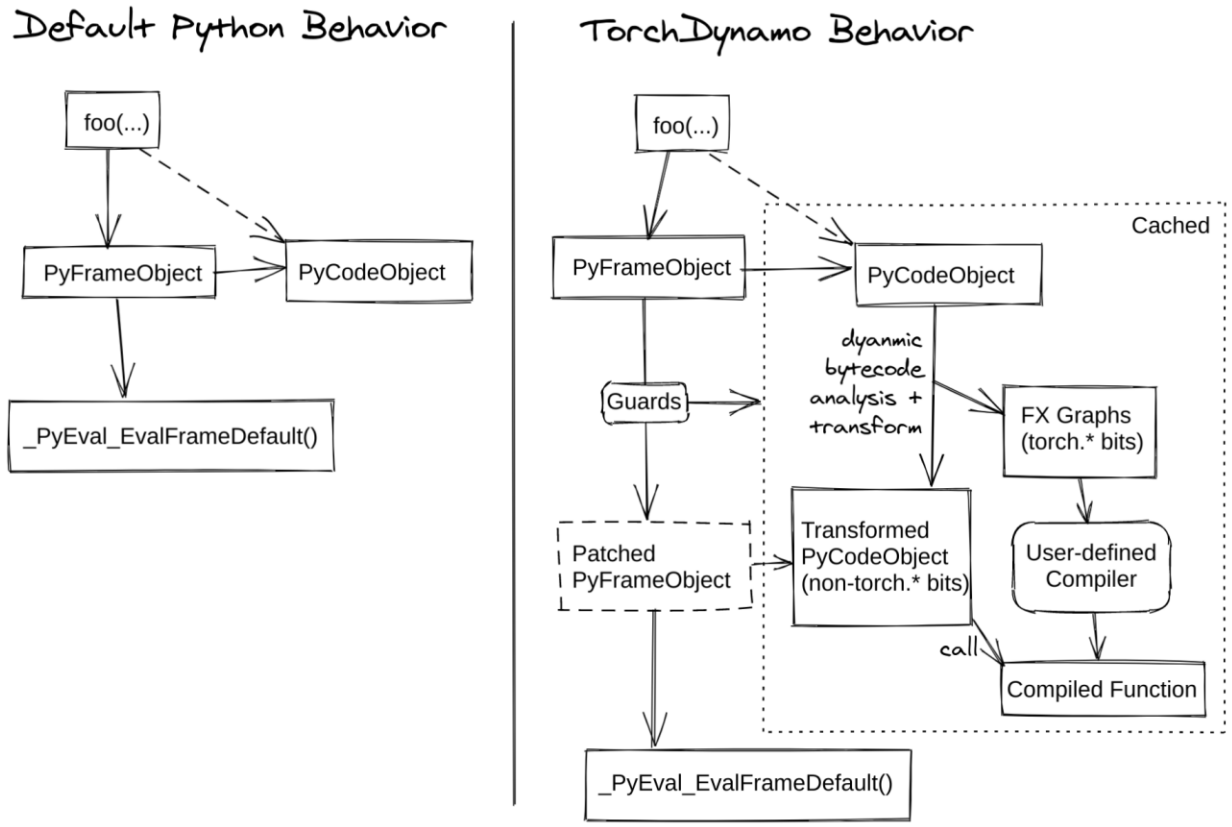
The torch compiler

- Compiles the GPU kernels for better efficientcy
- Very complex, do not worry

- Important: **less memory, more speed**

With native torch:
`torch.compile(model)` or
`model.compile()`

With `Trainer`:
`torch_compile=True` in the args

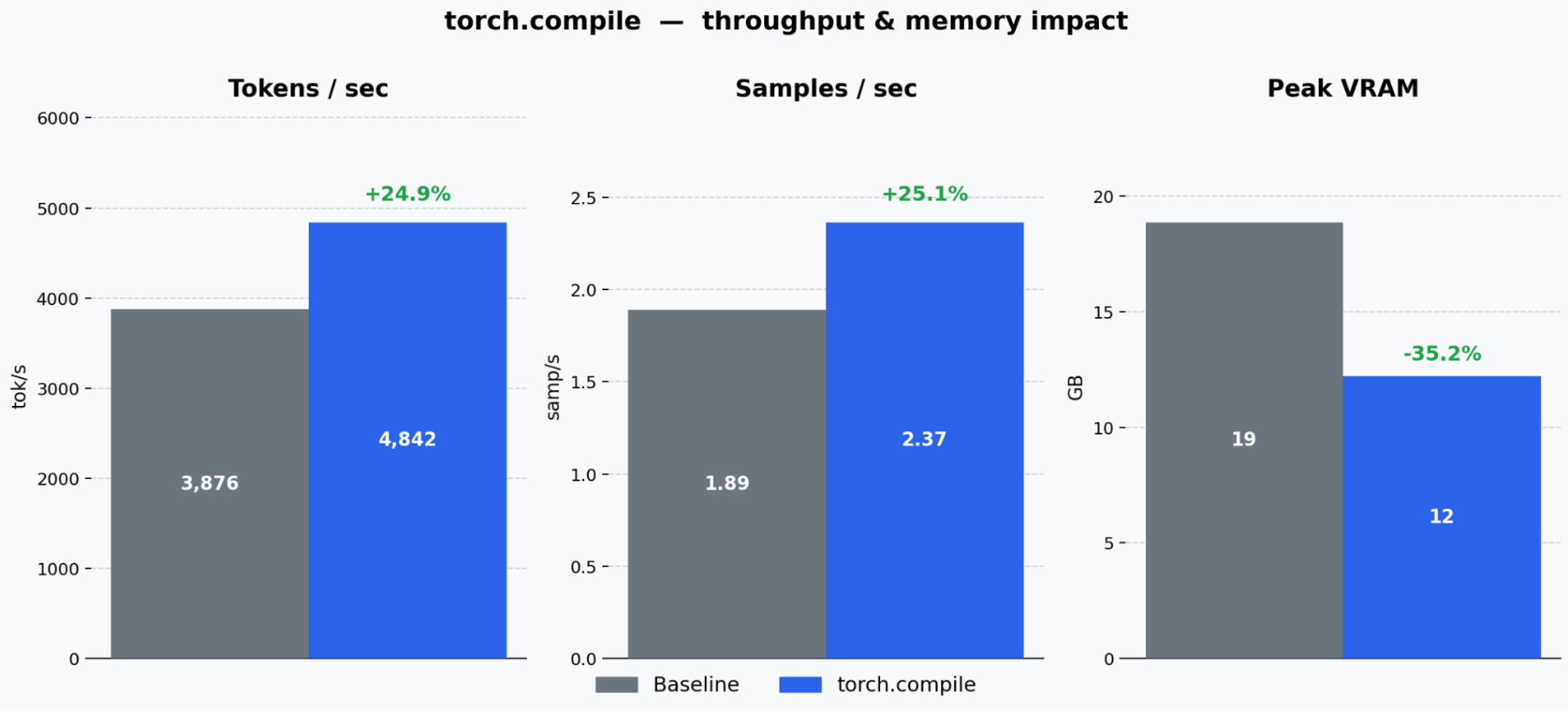


Credits (a blog in chinese): https://zhuanlan.zhihu.com/p/193839555645149286

The torch compiler

`torch.compile(model)`: one line of code
`torch_compile=True`: in the args

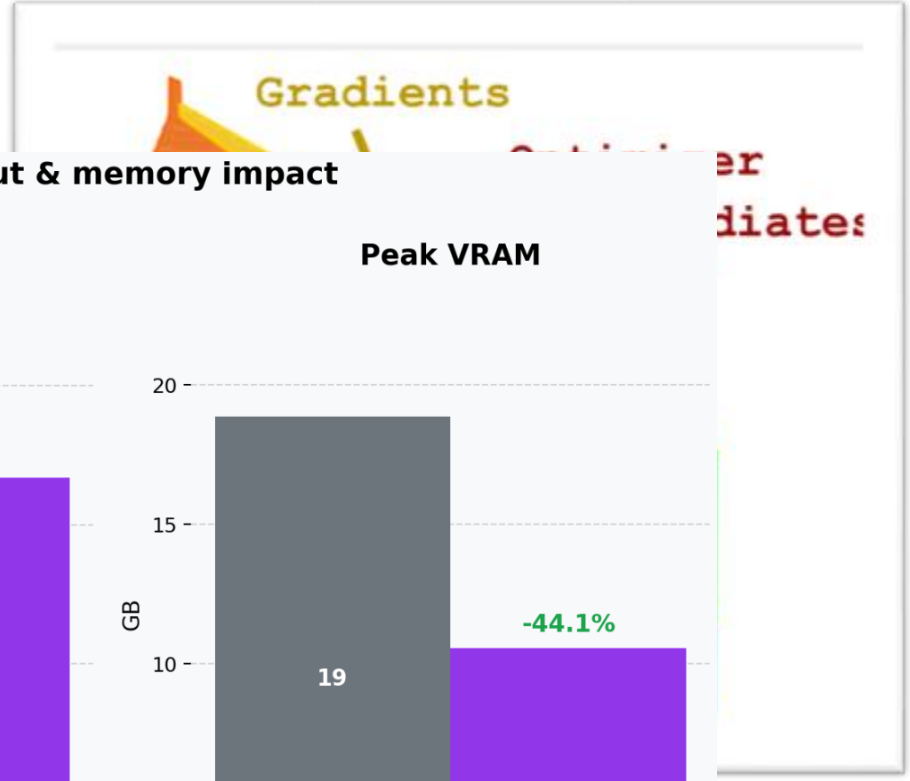
Results: **less memory, more speed**



Batch_size is 8 to better showcase the results

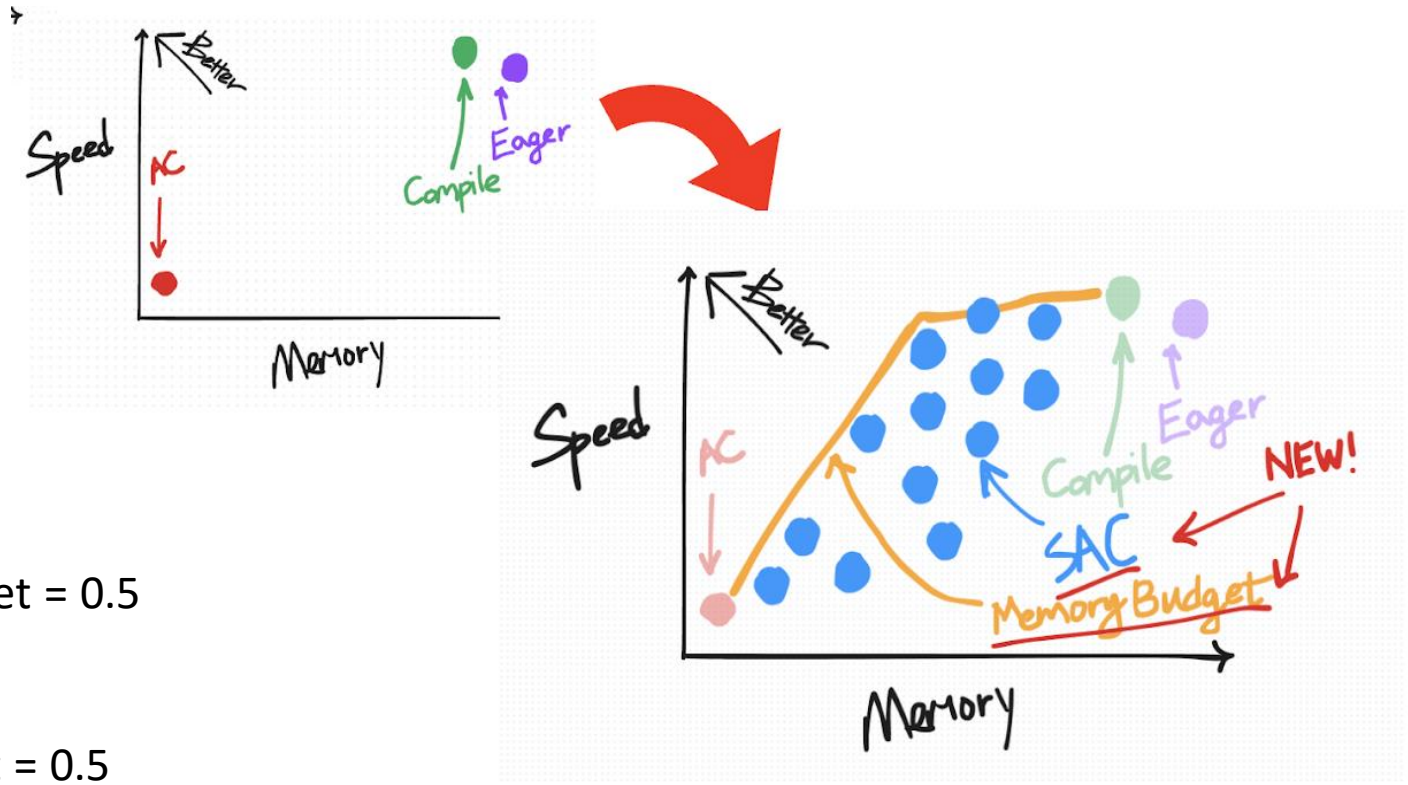
Gradient/Activation Checkpointing

- **Do not save the activations**
- BWD always recomputes what it needs
- **Huge memory saving with a speed decrease!**



Activation Checkpointing & Torch compile

- **Selective Activation Checkpointing**
- Save only the most compute intesive ops
- Let the compiler handle it



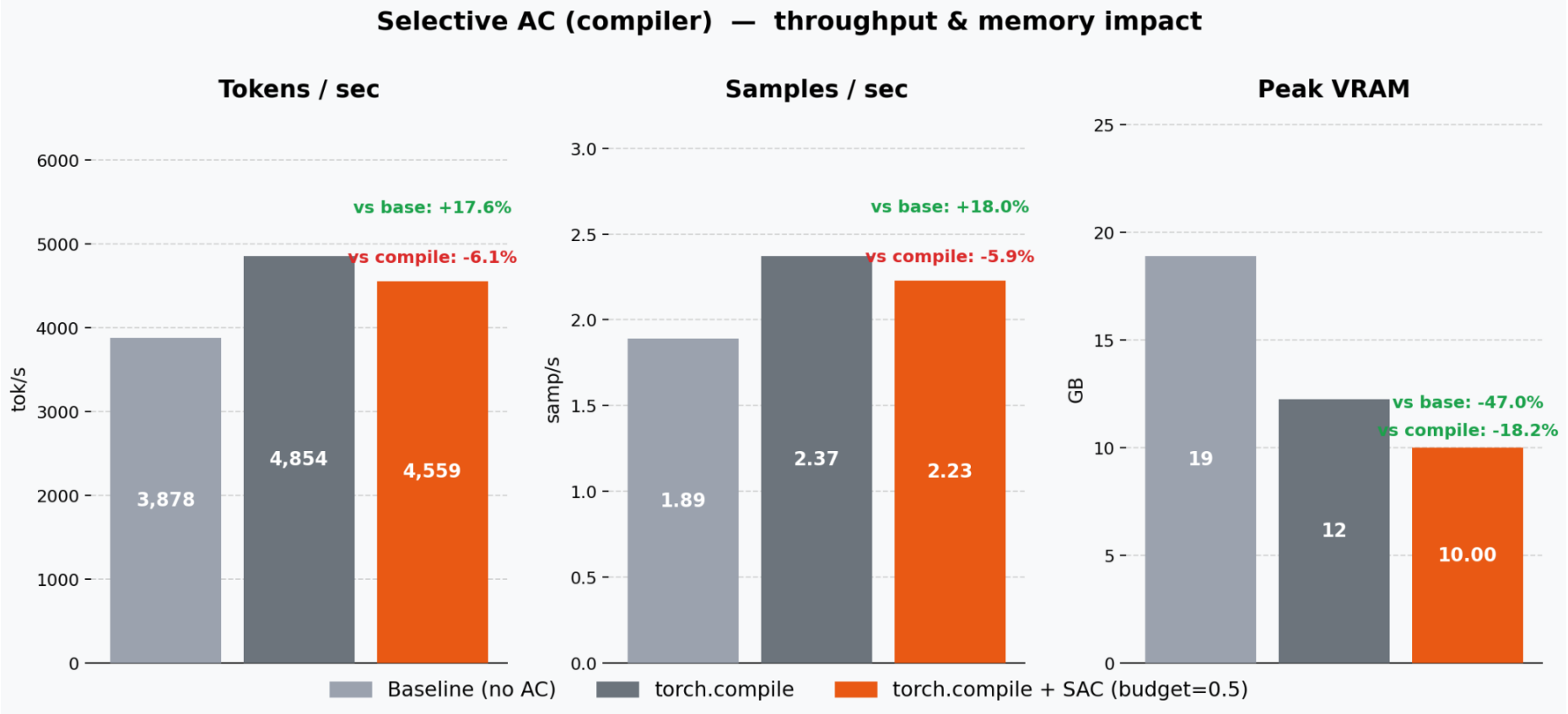
Add this on top of the file (and use compile):

```
torch._functorch.config.activation_memory_budget = 0.5

# torch<=2.10.0
torch._dynamo.config.activation_memory_budget = 0.5
```

Activation Checkpointing & Torch compile

torch._functorch.config.activation_memory_budget = 0.5

# torch<=2.10.0
torch._dynamo.config.activation_memory_budget = 0.5

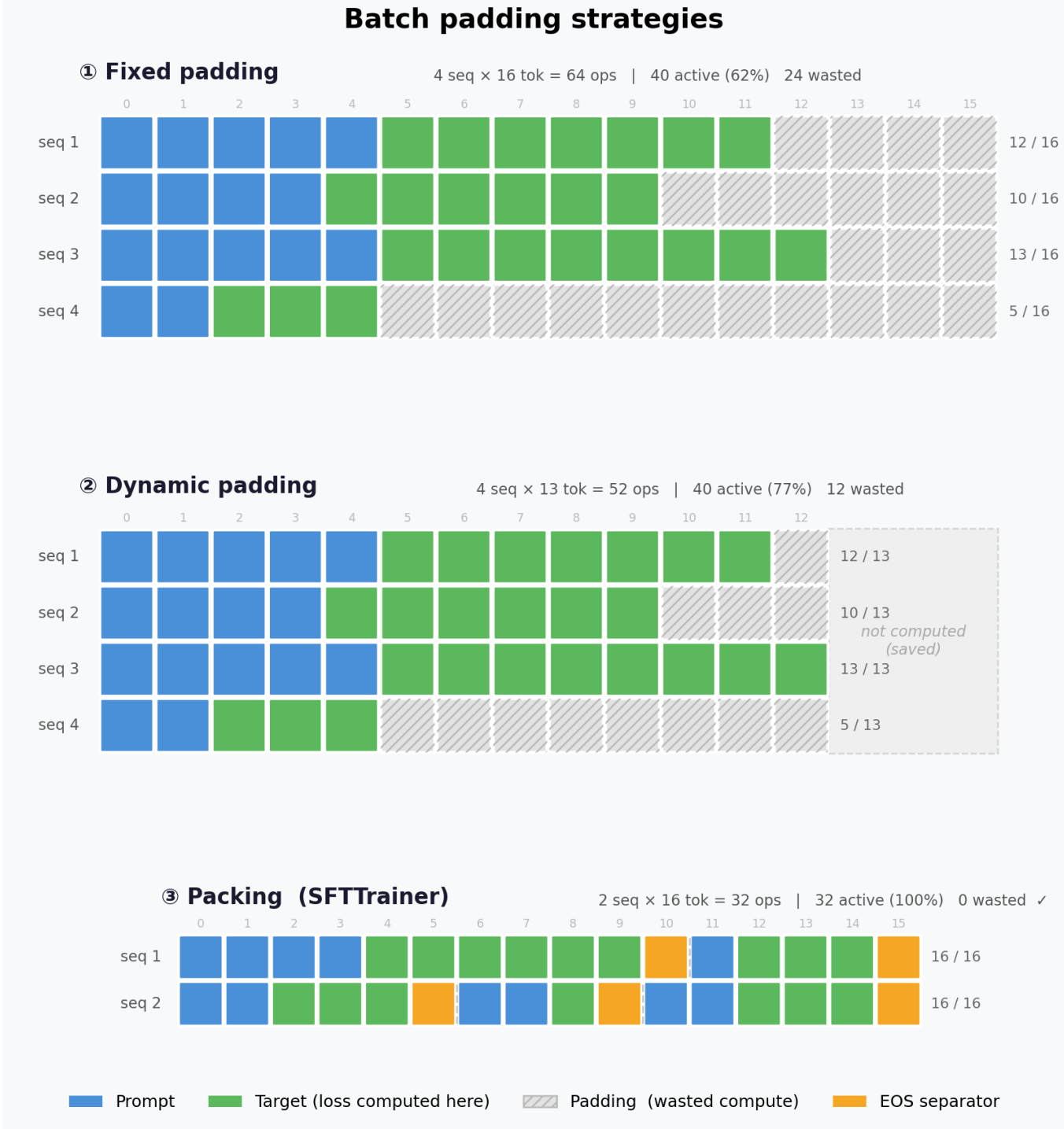


How to construnct your batch

Depending on the problem we may need to find a different balance

- Always fit as many samples as possible into the GPU
- Always **use the memory**! You dont pay for it
- Increase the batch size or the model size

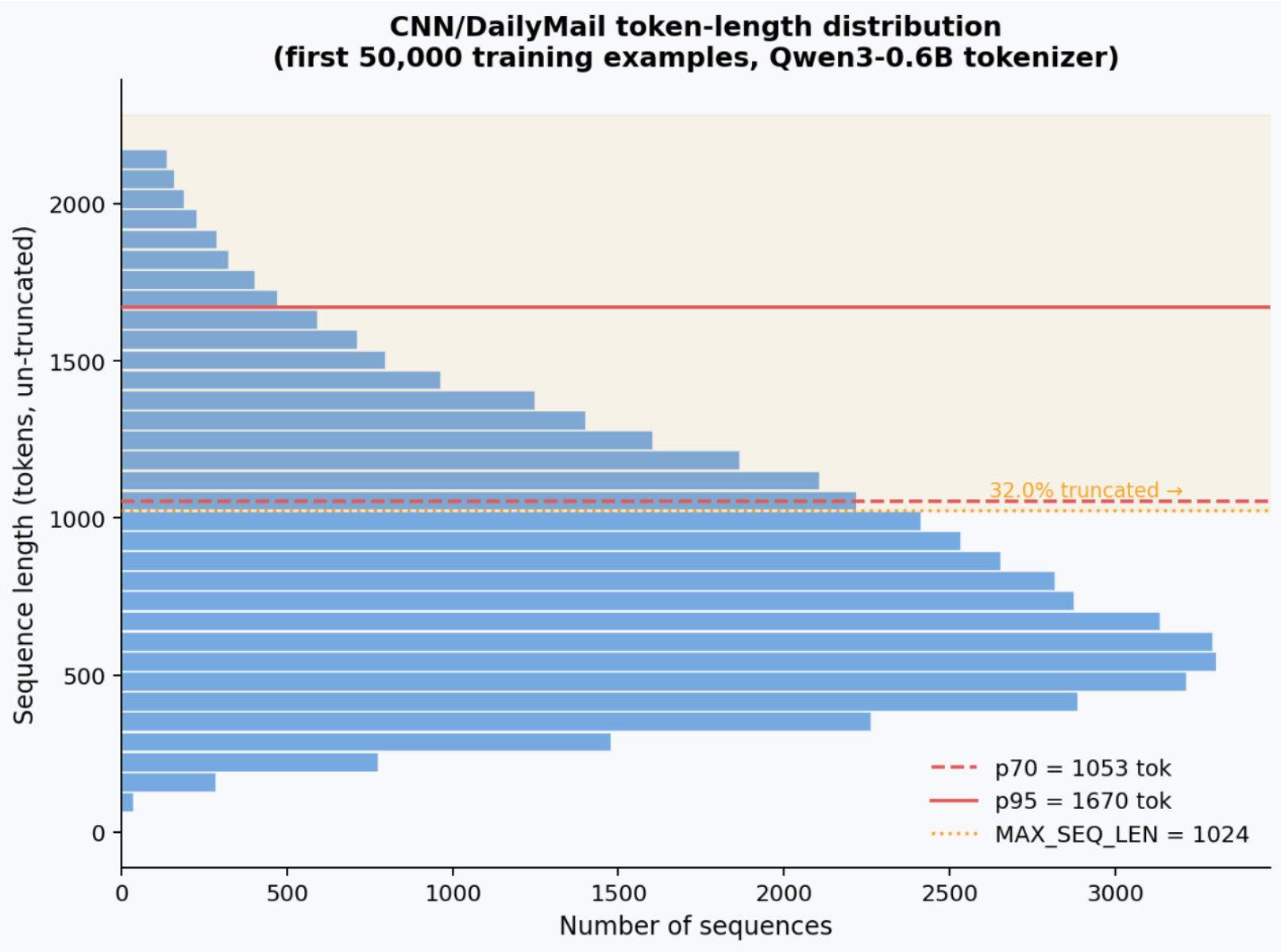
- When you cannot increase the batch size
  - You make the batch more efficient
  - Which there are different strategies to do so


Batch padding strategies
① Fixed padding
4 seq × 16 tok = 64 ops | 40 active (62%) 24 wasted
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
seq 1
seq 2
seq 3
seq 4
12 / 16
10 / 16
13 / 16
5 / 16
② Dynamic padding
4 seq × 13 tok = 52 ops | 40 active (77%) 12 wasted
0 1 2 3 4 5 6 7 8 9 10 11 12
seq 1
seq 2
seq 3
seq 4
12 / 13
10 / 13
not computed
(saved)
13 / 13
5 / 13
③ Packing (SFTTrainer)
2 seq × 16 tok = 32 ops | 32 active (100%) 0 wasted ✓
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
seq 1
seq 2
16 / 16
16 / 16
Prompt
Target (loss computed here)
Padding (wasted compute)
EOS separator

Dynamic vs Fixed Padding

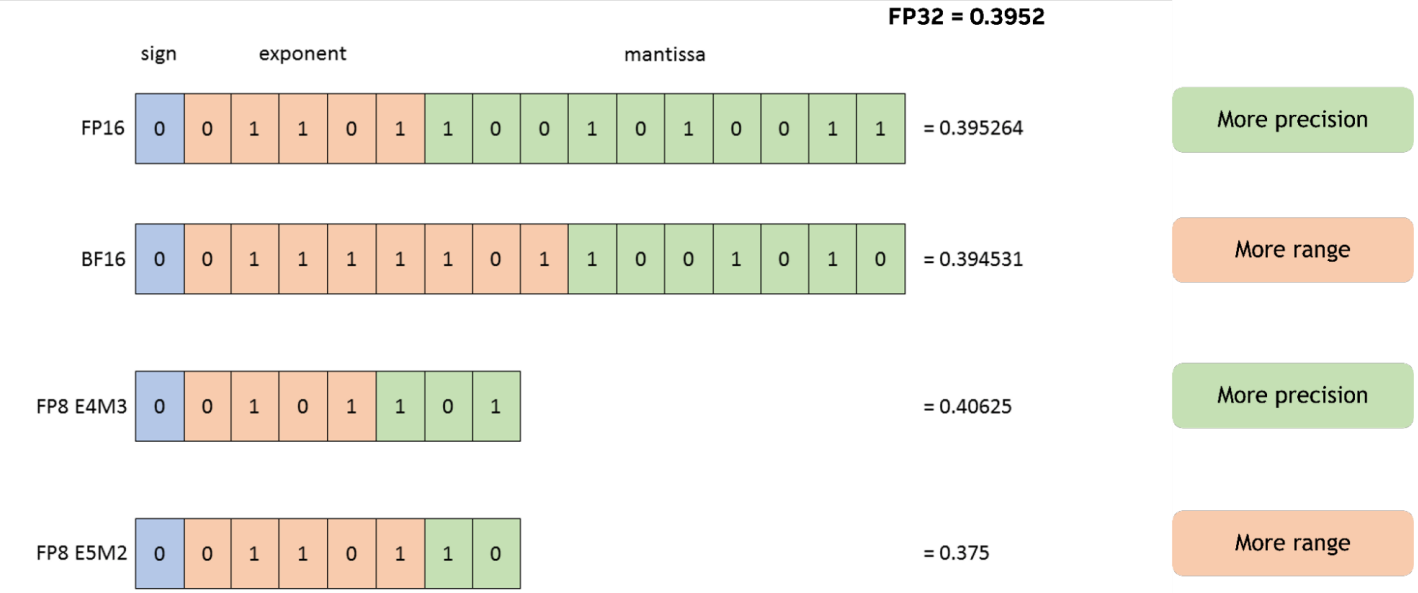
```
from transformers import (
    Trainer,
    DataCollatorForSeq2Seq,
)

collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model,
    padding="longest",
    pad_to_multiple_of=16, # for the Tensor Cores
    label_pad_token_id=-100,
)

trainer = Trainer(model, collator)
trainer.train()
```

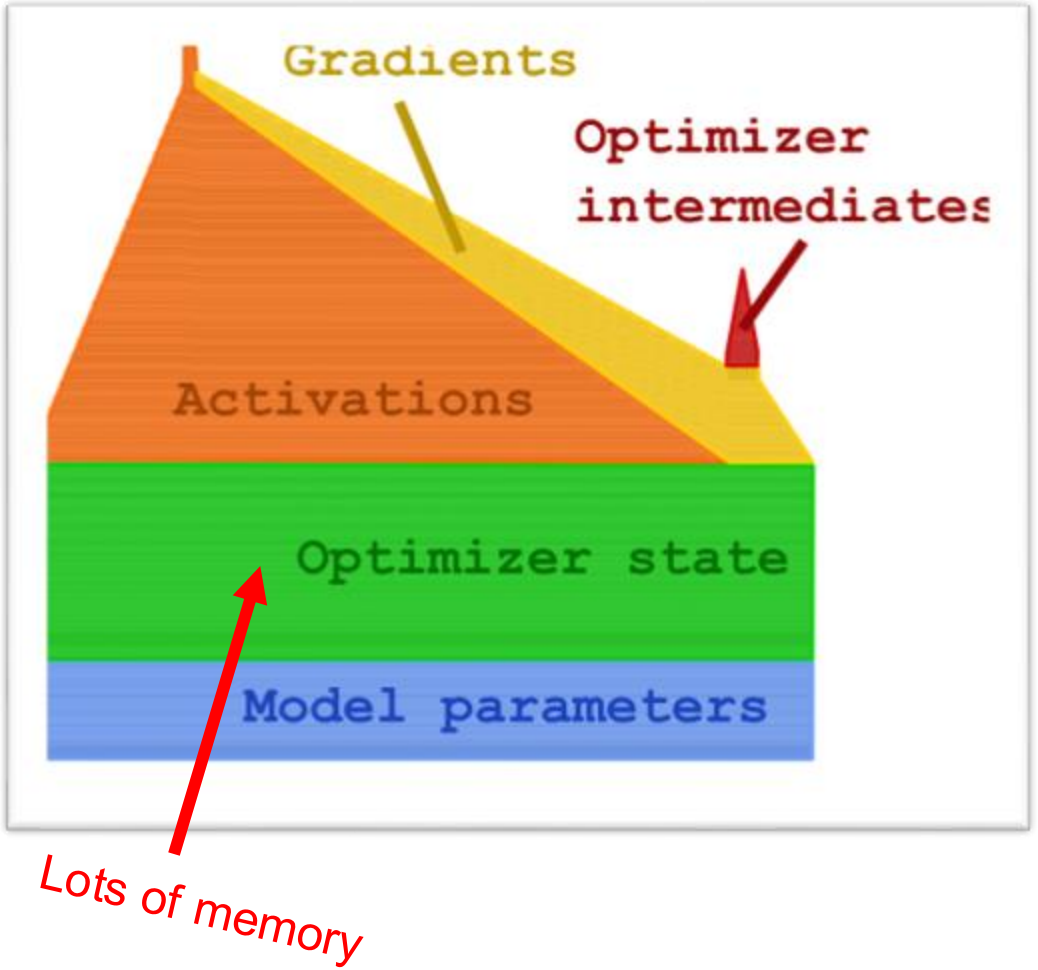
Look at the dataset



Different Optimizers

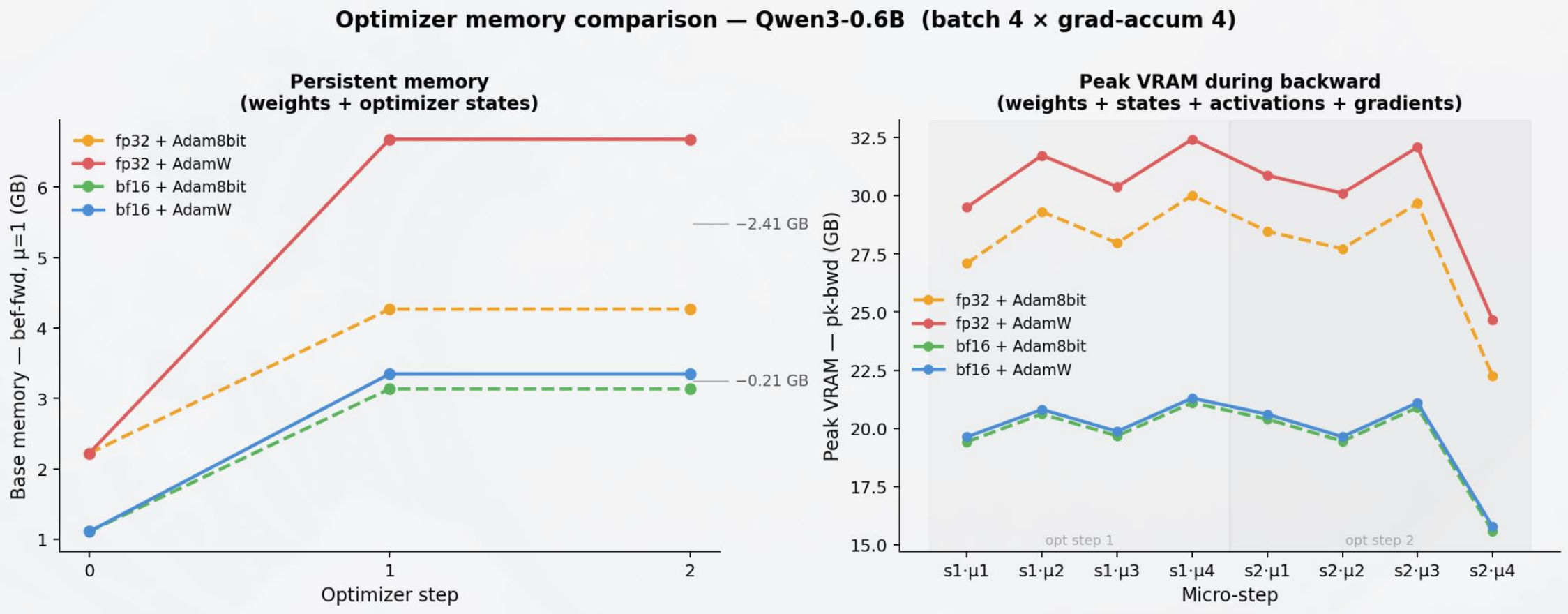
Compute in float32, save in float8





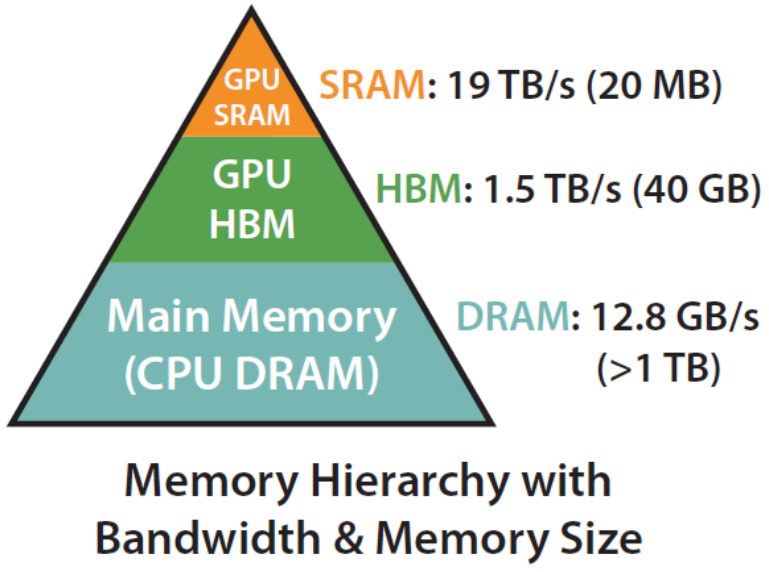
Note: usually the optimizer stores in float32

Different Optimizers

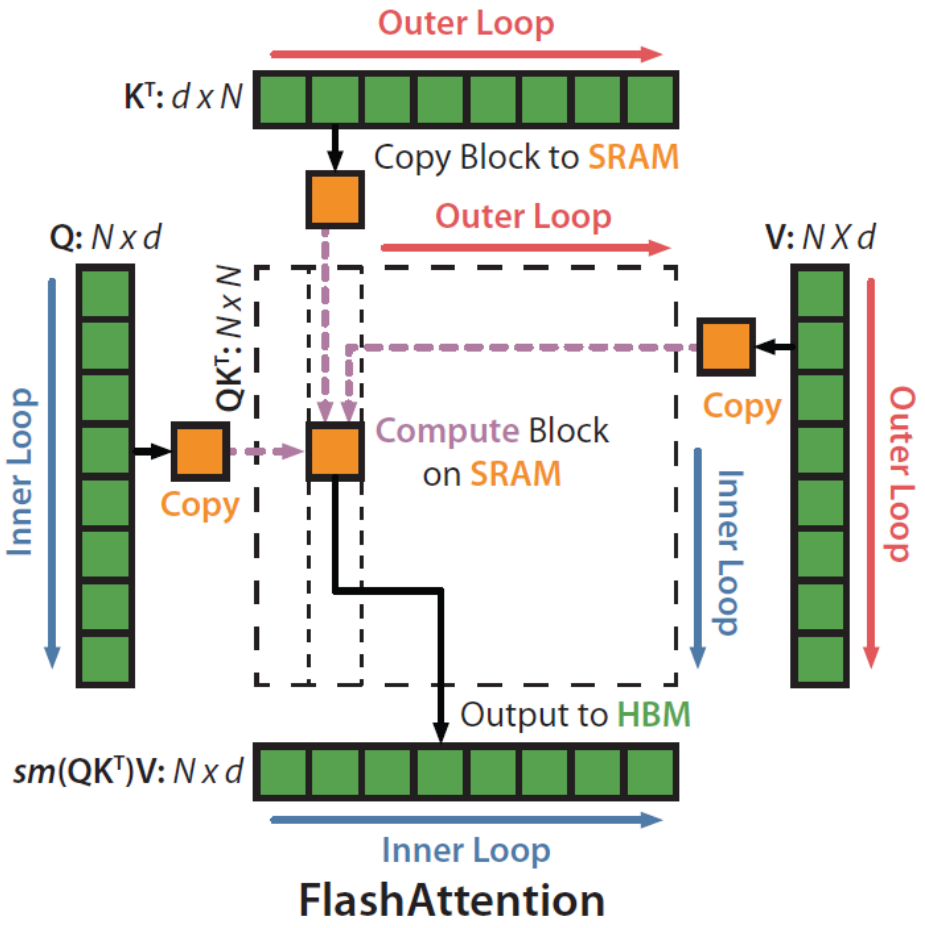


Flash Attention

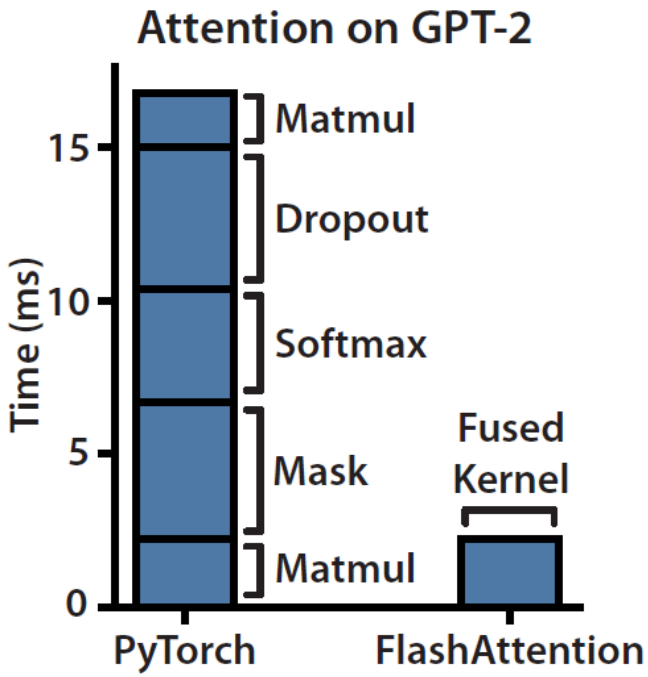
Hardware efficient Attention implementation (equivalent to MHA)



Memory Hierarchy with Bandwidth & Memory Size

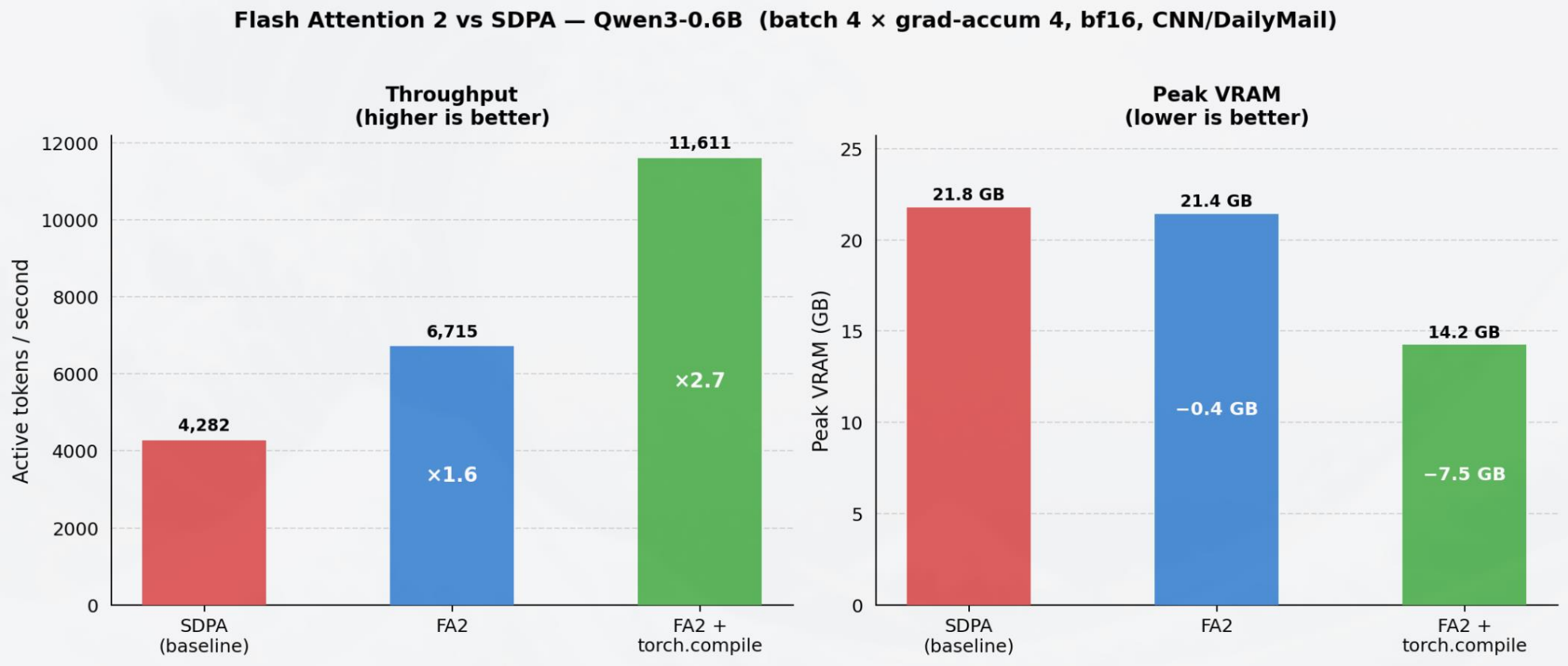


FlashAttention



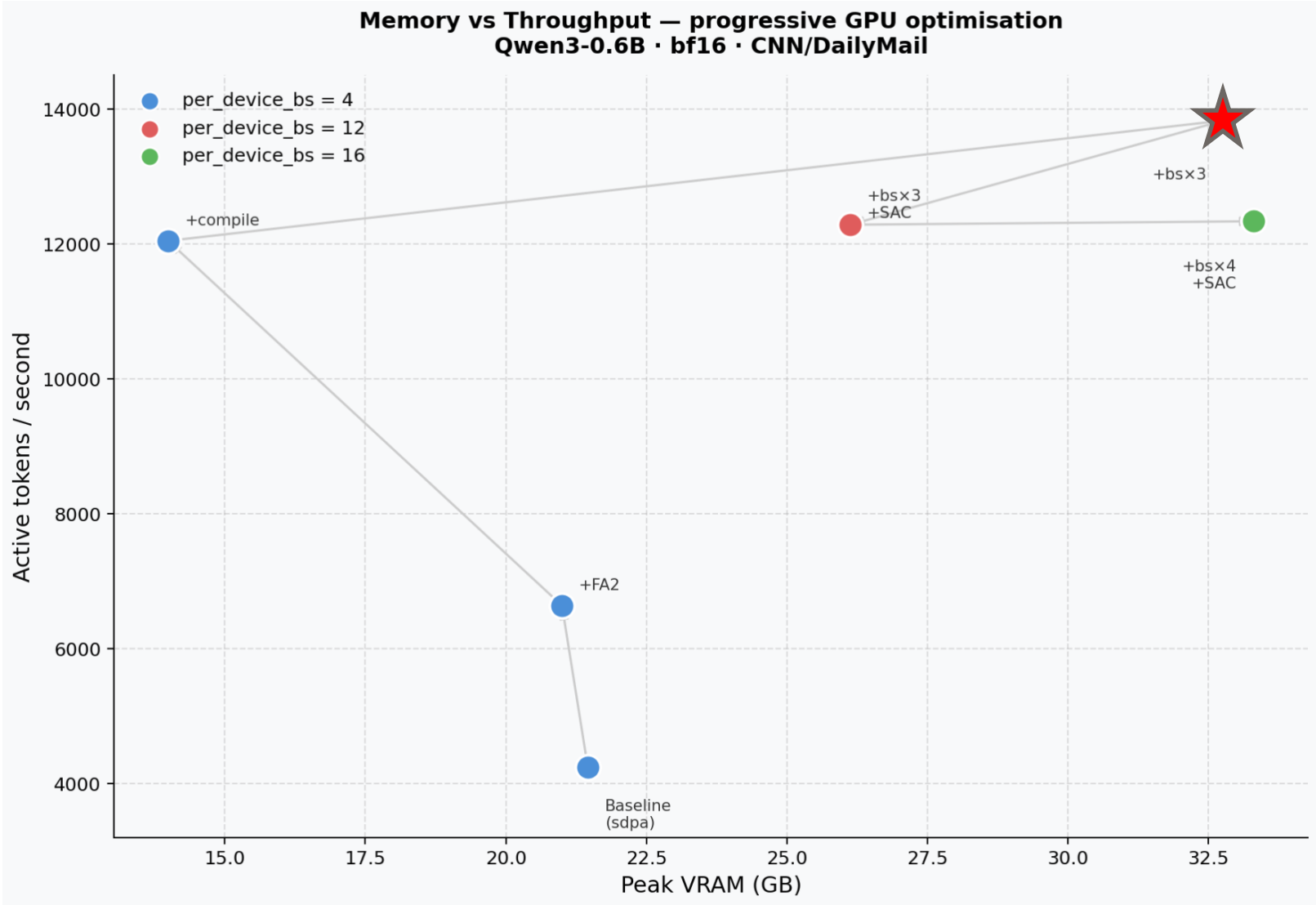
Flash Attention

```
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    dtype=torch.bfloat16,
    attn_implementation='flash_attention_2'
)
```

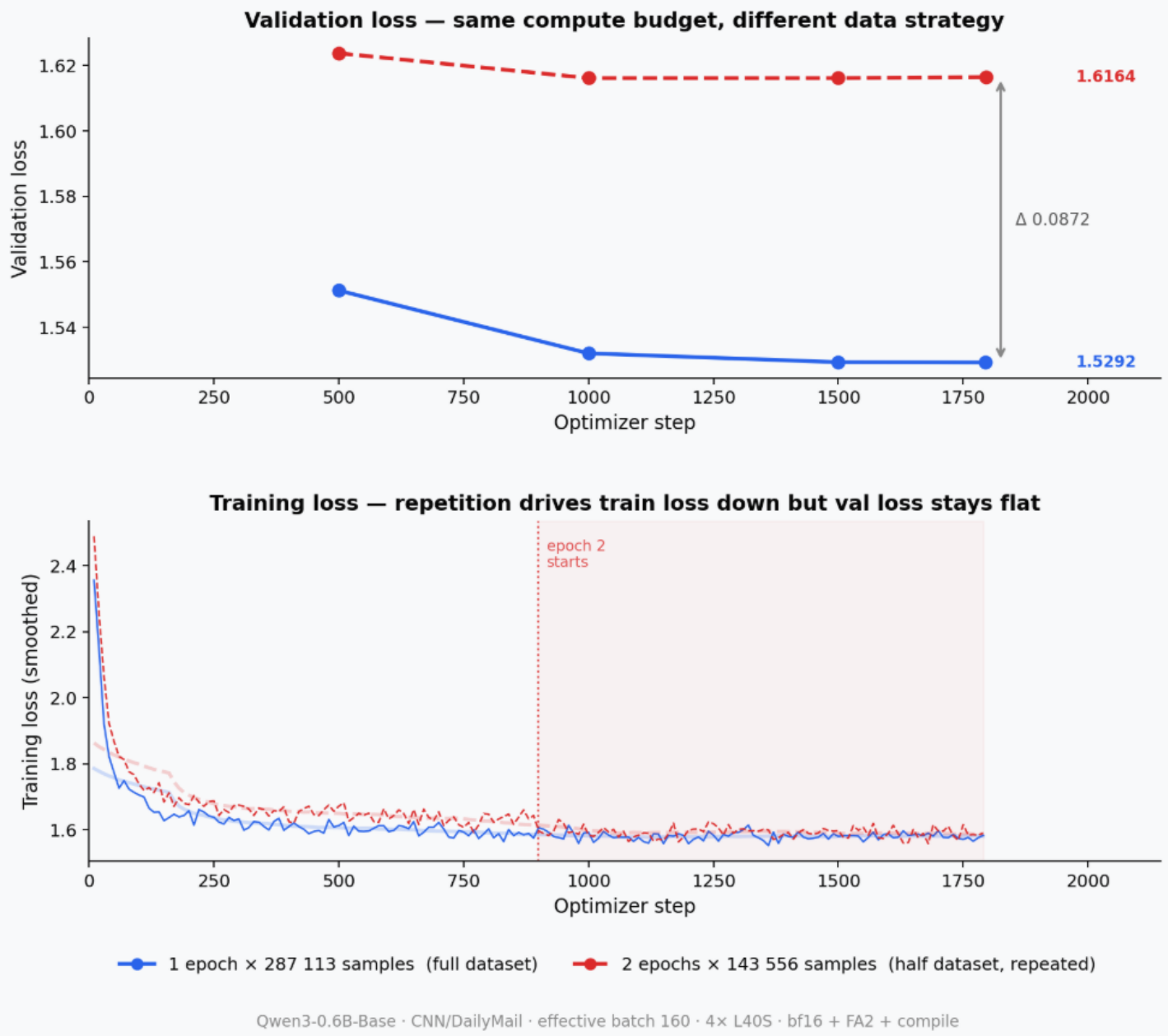


Putting everything together

1. Baseline (batch_size = 4)
2. FlashAttention
3. Compile
4. Increase batch_size to 12
5. Try SAC
6. Try to further increase batch_size



Too epoch or not to epoch?



Thank you for your FlashAttention!

Computer Vision Center
CVC

Edifici O, Campus UAB
08193 Bellaterra
(Cerdanyola)
Barcelona, Spain
www.cvc.uab.es